Development Plan

Production plan for the Tamil Nadu Government Agriculture Schemes RAG application.

Document purpose

This production-grade planning document defines scope, implementation controls, quality gates, and operating workflow for the Tamil Nadu Agriculture Schemes RAG application.

Attribute	Value
Application	Tamil Nadu Agriculture Schemes Assistant
Source page	https://www.tn.gov.in/scheme_list.php?dep_id=Mg==
Current local schemes	54
Current indexed chunks	317
Chat model	gpt-5.4-mini
Embedding model	text-embedding-3-small
Dataset hash prefix	603e7e7ae170

Prepared for implementation review, production readiness, and handoff.

1. Executive Summary

The application is a Streamlit-based retrieval assistant that helps users ask natural-language questions about publicly available Tamil Nadu Agriculture - Farmers Welfare Department schemes. The product combines a defensive scraper, structured local storage, FAISS vector retrieval, OpenAI models, and LangSmith observability.

Primary production objective

Deliver grounded, source-backed scheme answers while preventing homepage/navigation text, stale indexes, missing keys, or unverified model knowledge from polluting responses.

2. Product Scope

Area	In scope	Out of scope
Users	Farmers, student reviewers, administrators, demo evaluators	Official government service obligations
Data	Public scheme pages from approved tn.gov.in domains	Private data, protected endpoints, personal data scraping
Answers	Grounded answers from retrieved chunks with source URLs	Legal, financial, or eligibility guarantees beyond source data
Operations	Manual refresh, rebuild, troubleshooting, local artifacts	Fully managed cloud deployment unless added later

3. Target Architecture

Streamlit UI	->	Scraper	->	JSON/CSV	->	Documents	->	FAISS	->	Retriever	->	Chat Model

- Configuration is centralized in config.py and all secrets remain in environment variables.
- scraper.py owns website access, normalization, deduplication, and safe persistence.
- rag_pipeline.py owns document formatting, chunking, dataset hashing, FAISS lifecycle, and answer generation.
- app.py owns the Streamlit interface, user workflow, buttons, session state, source display, and data browsing.

4. Delivery Roadmap

Phase	Goal	Key deliverables	Exit criteria
P0 - Foundation	Create runnable local app	Config, requirements, README, Streamlit shell	App starts and displays friendly missing-key messages
P1 - Data ingestion	Scrape reliable public data	Session handling, retries, redirect detection, JSON/CSV	Valid schemes saved; failed refresh preserves old data
P2 - Retrieval	Build reusable vector index	Documents, chunking, FAISS, metadata validation	Index reuses when valid and rebuilds when dataset changes
P3 - Answering	Grounded assistant	Strict prompt, source list, unavailable-information behavior	No model call when no relevant context exists
P4 - UX hardening	Production-grade interface	Readable controls, responsive layout, chat state, tab placement	Buttons visible; no contrast or overflow defects

Phase	Goal	Key deliverables	Exit criteria
P5 - Observability	Trace critical runs	LangSmith project metadata and tags	Tracing works when keys are configured; secrets excluded
P6 - Release	Operational handoff	Runbook, PDFs, acceptance checklist	User can refresh, rebuild, ask, and troubleshoot locally

5. Workstreams And Owners

Workstream	Responsibilities	Quality controls
Frontend	Streamlit page structure, sidebar actions, chat session state, source expanders, CSV download	Browser checks across desktop and mobile; contrast and layout review
Scraper	Approved-domain crawling, retries, cleaning, deduplication, preservation of last good dataset	Homepage detection tests; no overwrite with empty data
RAG	Document conversion, chunking, dataset hashing, FAISS persistence, retrieval, answer synthesis	Index metadata validation; strict answer contract
Security	Secret handling, safe local FAISS loading, no unsafe scraped HTML	No API keys in UI/logs; explicit dangerous-deserialization boundary
Observability	LangSmith tracing and safe metadata	Tracing can be disabled; no cookies or headers in metadata
Documentation	Setup, troubleshooting, architecture, workflow, limitations	Windows and macOS/Linux commands; known redirect issue documented

6. Engineering Backlog

Priority	Item	Production rationale
High	Add unit tests for homepage detection and scheme normalization	Prevents accidental indexing of navigation/footer text
High	Add smoke test script for scrape -> index -> ask	Makes demo readiness repeatable
High	Improve selector regression fixtures from saved HTML samples	Protects against website layout drift
Medium	Add admin-only refresh guard or password for shared deployments	Prevents unplanned scraping from public users
Medium	Add offline demo dataset mode	Keeps demos stable when website blocks or redirects
Medium	Add answer feedback capture	Supports relevance tuning and retrieval quality improvement
Low	Add multilingual answer mode	Improves accessibility for Tamil-speaking users when model support is configured

7. Testing Strategy

Layer	Checks	Pass signal
Static	py_compile, dependency import checks, secret placeholder checks	No syntax or import errors
Scraper	Redirect, homepage marker, approved host, empty data, dedupe, previous data preservation	Only valid scheme records persisted
Index	Dataset hash, model name, chunk size, chunk overlap, index file existence	Rebuild only when required
RAG	Known scheme question, unrelated question, source dedupe, strict unavailable answer	Sources included and no unsupported claims
UI	Desktop and mobile rendering, action buttons, chat input, tabs below output, data browser	No overlap, poor contrast, or broken state
Ops	Restart, clear cache, refresh, rebuild, direct python app.py launch	Operator can recover without editing code

8. Release Gates

- OPENAI_API_KEY is configured outside source control.
- LANGSMITH_TRACING is either validly configured or disabled.
- Scrape refresh produces non-empty JSON and CSV.
- FAISS metadata matches dataset hash, embedding model, and chunk settings.
- At least five representative questions return grounded answers with sources.
- One unrelated question returns the approved unavailable-information response.
- All action buttons are visible and readable in the supported viewport.
- README, workflow PDF, and development plan PDF are delivered.

9. Risk Register

Risk	Impact	Mitigation	Owner
Government page redirects to homepage	Scraper could index unrelated content	Detect final URL and homepage markers; preserve previous dataset	Scraper
HTML layout changes	Scheme extraction quality drops	Selector fallbacks and HTML fixture tests	Scraper
OpenAI key missing or invalid	Chat and indexing fail	Friendly validation and setup docs	Config/UI
FAISS pickle loading risk	Unsafe deserialization if untrusted index used	Load only app-generated local index and document boundary in code	RAG
Model hallucination	Incorrect eligibility or benefit claims	Strict prompt, no outside knowledge, source display	RAG
Stale local data	Outdated answers	Expose scrape timestamp and refresh control	Operations
Rate limiting or blocking	Refresh failures	Polite headers, session warmup, retries, request delay	Scraper

10. Operational Readiness

Runbook summary

Start with python app.py or streamlit run app.py. Refresh data only when needed, rebuild FAISS after every successful refresh, verify a sample answer, and keep generated data and indexes local unless a secure deployment plan is added.

Operation	Command or action	Expected result
Launch	python app.py	Streamlit opens in the browser
Refresh	Sidebar -> Refresh Website Data	New JSON and CSV are saved only after valid scrape
Rebuild	Sidebar -> Rebuild FAISS Index	index.faiss, index.pkl, and metadata are updated
Troubleshoot	Streamlit menu -> Clear cache / Rerun	UI state and cached data can be refreshed
Stop	Ctrl + C in terminal	Local server shuts down